

API Design and Export Contracts Guide

Stable boundaries for traceability data and partner integrations

Document ID	ENG-API-02
Status	Approved
Effective date	2026-09-15
Owners	Trace Platform and Partner Integrations
Applies to	HTTP APIs, downloads, and machine-readable exports

1. Contract first

An API contract is the externally observable agreement between a producer and its consumers. Begin with consumer workflows, resource ownership, authorization, data sensitivity, and failure behavior. Implementation convenience does not justify an unstable public shape.

Required: A new or changed partner-facing endpoint must have an OpenAPI description, representative examples, boundary validation, authorization rules, and contract tests before release.

2. Resources and URLs

- Use nouns for resources and collections: `/origin-passports`, `/batches/{batchId}`, and `/shipments/{shipmentId}`.
- Use stable opaque identifiers in URLs. Do not expose database position or mutable display names as identity.
- Represent a workflow action as a subordinate resource when it has its own result or lifecycle, such as `POST /origin-passports/{id}/exports`.
- Avoid verbs in collection paths unless the operation cannot be modeled honestly as resource creation or transition.
- Keep nesting shallow. A URL should identify ownership, not reproduce the entire data model.

3. HTTP behavior

Operation	Expected behavior	Typical success
GET	Read without changing server state.	200 or 304
POST	Create a resource or initiate a non-idempotent operation.	201, 202, or 200
PUT	Replace a complete resource at a known URL; idempotent.	200 or 204
PATCH	Apply a documented partial update with explicit field semantics.	200 or 204

Operation	Expected behavior	Typical success
DELETE	Remove or retire according to retention policy; idempotent outcome.	204

Use 400 for malformed requests, 401 when authentication is absent or invalid, 403 when the caller is known but not permitted, 404 when the resource is unavailable to the caller, 409 for state conflicts, 422 for well-formed input that fails domain validation, and 429 for rate limits.

4. JSON representation

- Use UTF-8 JSON with consistent lower camel case property names unless an established API in the repository uses another convention.
- Return ISO 8601 timestamps with an explicit offset, preferably UTC Z. Return calendar-only business dates as YYYY-MM-DD.
- Do not overload missing, null, empty string, and zero. Document each field's absence semantics.
- Represent money as a decimal string plus an ISO currency code. Never use binary floating point for financial calculations.
- Additive fields are normally backward compatible, but consumers must be told to ignore fields they do not understand.
- Do not expose internal notes, credentials, personal contact details, negotiated pricing, or operational identifiers unless the contract and authorization explicitly require them.

5. Errors

Errors use one stable envelope and a machine-readable code. Human-readable messages may improve without requiring a client release; client logic must branch on the code, not message text.

```
{
  "error": {
    "code": "export_format_not_supported",
    "message": "The requested export format is not available.",
    "requestId": "req_01J...",
    "details": [{"field": "format", "reason": "allowed: json"}]
  }
}
```

Do not return stack traces, dependency messages, secrets, or existence details that bypass authorization. Log diagnostic context under the same request identifier.

6. Collections and pagination

- Paginate collections that can grow. Prefer an opaque continuation cursor over a mutable row offset.
- Document stable ordering and apply a deterministic tie-breaker.
- Bound page size and return links or cursor fields needed for the next request.
- Filters and sort keys form part of the contract. Validate unsupported combinations rather than silently ignoring them.

7. Idempotency and concurrency

- Support an idempotency key for partner operations that may be retried and would otherwise create duplicates.
- Store enough request identity to return the original outcome for an accepted retry.
- Use entity tags or an explicit version for conflicting edits where lost updates matter.
- Do not promise exactly-once delivery across distributed systems. Design repeatable outcomes and observable reconciliation.

8. Export contracts

A download is an API response and receives the same contract discipline as JSON. The requested format determines serialization, filename, media type, escaping, and validation. Renaming a JSON file to end in `.csv` is not CSV support.

Format negotiation

- Use a documented query parameter or format-specific subordinate resource when users explicitly choose a download format.
- Return 406 or a domain validation error for an unsupported representation; do not silently substitute another format.
- Set `Content-Type` and `Content-Disposition` consistently with the bytes returned.

CSV requirements

- Generate CSV with a standards-based library, not string concatenation.
- Use UTF-8, a stable documented header order, and one record shape per contract version.
- Quote commas, quotes, carriage returns, and line breaks correctly; double embedded quotes.
- Represent absent optional values as empty fields without dropping columns.
- Protect spreadsheet consumers from formula injection when an exported text field begins with a formula-significant character.
- Test parsing the produced bytes with an independent CSV parser and assert headers, row count, and representative edge cases.

9. Origin Passport example

Origin Passports are immutable issued snapshots. Adding CSV support should serialize the same authorized snapshot currently used by JSON; it must not rebuild historical values from mutable source tables or create a second business-data pathway.

```
GET /origin-passports/OP-EC-2026-0042/export?format=csv
```

```
200 OK
```

```
Content-Type: text/csv; charset=utf-8
```

```
Content-Disposition: attachment; filename="OP-EC-2026-0042.csv"
```

The endpoint must preserve existing authorization, supersession visibility, audit logging, and retention behavior. A format change does not authorize additional fields.

10. Versioning and deprecation

- Prefer compatible additions and explicit capability discovery before introducing a new API version.
- Version when an existing consumer cannot continue safely, not for every implementation change.
- Publish a migration path and retirement date before removing a contract. Monitor usage and contact known consumers.
- Keep old and new behavior covered by contract tests throughout the supported overlap.

11. Security and operations

- Authorize the requested resource and action on every request. Authentication alone is not authorization.
- Apply least privilege, bounded inputs, rate limits, request-size limits, and safe logging.
- Record request identifiers, outcomes, latency, dependency failures, and domain-relevant audit events without recording sensitive payloads.
- Define alerts for sustained error rates, latency, and export failures. Document rollback or feature-disable steps before broad release.

12. Contract review checklist

1. Consumer workflow and ownership are explicit.
2. Schema, examples, nullability, and sensitive-data exclusions are documented.
3. Status codes and error codes are predictable.
4. Retries, concurrency, pagination, and versioning are addressed where relevant.
5. Authorization and operational telemetry are tested.
6. Contract tests prove the actual bytes, headers, and failures a consumer receives.